Concept Questions

C++ Operator Overloading – Complex Numbers

1 Why can't operator<< for `cout << x` be a member function of Complex?

Answer: Because the left side of << is cout (type std::ostream), not a Complex.

- A member function always works on the object on its **left**. A member operator<< in Complex would have to be called as `x << cout`, which is backwards.
- We cannot add a function to std::ostream because it is a library class.
- So operator<< must be a **non-member** function (often a friend if it needs private data).

```
std::ostream& operator<<(std::ostream& os, const Complex& c);
```

2 Why does operator<< return std::ostream& instead of void?

Answer: So that output can be chained: `cout << x << " " << y;`

- That line runs as `((cout << x) << " ") << y`. Each << must give back the stream so the next << can use it.
- If it returned void, the next step would be `void << " "`, which does not compile.
- It returns a **reference** because a stream cannot be copied, and we want to keep using the same stream.

3 Why is the stream passed by non-const reference but the complex number by const reference?

Parameter	How	Why
`std::ostream& os`	non-const reference	Printing **changes** the stream (its buffer and state), so it must not be const. It also cannot be copied.
`const Complex& c`	const reference	We only **read** the number, never change it. A reference avoids a copy, and const lets us print constant objects and temporaries.

4 Why does postfix ++ have an extra int parameter?

Answer: It is a dummy parameter that tells the compiler this is the postfix version.

- Prefix and postfix are both named operator++, so they need different parameter lists to be told apart.
- The int is never used. The compiler just passes 0.

```
Complex& operator++();      // prefix  : ++x
Complex  operator++(int);   // postfix : x++
```

5 Why must prefix ++ return a reference but postfix ++ return by value?

	Returns	Reason
Prefix ++x	reference (Complex&)	It increments the object and returns **the same object**. No copy is needed.
Postfix x++	value (Complex)	It must return the **old** value. So it saves a copy, increments the original, and returns the copy. That copy is a local variable and is destroyed when the function ends, so returning a reference to it would be a **dangling reference**.

```
Complex& operator++() {      // prefix
    ++real;
    return *this;           // the same object
}
Complex operator++(int) {   // postfix
    Complex old = *this;    // save old value
    ++real;
    return old;             // return the copy
}
```

6 What is printed for Complex x holding 1 + 1i and the statement `cout << x++ << " " << x;`?

Answer: `1 + 1i 2 + 1i`

- x++ returns the **old** value, so **1 + 1i** is printed first.
- After that, x becomes **2 + 1i** (assuming ++ adds 1 to the real part).
- Then a space is printed, followed by the new x: **2 + 1i**.

Note: Since C++17 this left-to-right order is guaranteed.

7 Which is more efficient for a struct, ++x or x++, and why?

Answer: ++x (prefix) is more efficient.

- `++x` changes the object directly and returns a reference. **No copy**.
- `x++` must make a **temporary copy** of the old value, increment, and return the copy.
- For simple types like int the compiler makes them equal, but for structs and classes the copy costs time. So use `++x` when you do not need the old value.

8 This version of operator<< has a bug. What does it print for 1 - 2i, and how would you fix it?

Buggy version:

```
std::ostream& operator<<(std::ostream& os, const Complex& c) {
    os << c.real << " + " << c.imag << "i";
    return os;
}
```

It prints: `1 + -2i` **(wrong)**

Why: it always prints a "+", even when the imaginary part is negative, so we get "+ -2i" instead of "- 2i".

Fix: check the sign of imag. If it is negative, print " - " and then the positive value (`-c.imag`).

```
std::ostream& operator<<(std::ostream& os, const Complex& c) {
    if (c.imag >= 0)
        os << c.real << " + " << c.imag << "i";
    else
        os << c.real << " - " << -c.imag << "i";
    return os;
}
```

Corrected output: `1 - 2i`